

ANALISANDO SNAP

Ana Carolina Balbino

Table of contents

Introdução	4
Começando	4
Instalação Local	4
Créditos	5
I Artigo	6
1 Análise Estrutural e Algorítmica da Rede Social LiveJournal	7
2 Cálculo de Determinação do Grafo a ser Utilizado	8
3 Descrição do Grafo Original	9
4 Metodologia	11
4.1 Dataset e Formato de Entrada	11
4.2 Ambiente Computacional	11
4.3 Procedimento de Análise Estrutural	11
4.4 Avaliação de Algoritmos	12
5 Resultados Obtidos	13
5.1 Tratamento dos Dados	13
5.2 Análise Estrutural do Grafo Completo	13
5.3 Desempenho dos Algoritmos	14
5.4 Propriedade Small-World	16
5.5 Análise de Lei de Potência (Power Law)	16
5.6 Robustez da Rede	17
6 Discussão Crítica	19
6.1 Decisões de Projeto e Viabilidade	19
6.2 Limitações Metodológicas e Estruturais	19
7 Conclusão	21
Agradecimentos	21
Apêndice	23
7.1 Repositório e Instruções de Execução	23

II	Scripts	24
8	Análise Estrutura - Comunidades	25
8.1	Tratamento da rede	25
8.2	2. Pre-processing	26
8.3	3. Report basic metrics	26
8.4	4. Componentes Conexas	27
8.5	5. Diâmetro e Raio	27
8.6	6. Comprimento medio dos caminhos	28
8.7	7. Coeficiente de clusteriação medio	29
8.8	8. Número de triângulos	29
8.9	9. Resumo	30
9	Análise Estrutura - Grafo Completo	32
9.1	Tratamento da rede	32
9.2	2. Pre-processing	33
9.3	3. Report basic metrics	33
9.4	4. Componentes Conexas	34
9.5	5. Diâmetro e Raio	34
9.6	6. Comprimento medio dos caminhos	35
9.7	7. Coeficiente de clusteriação medio	36
9.8	8. Número de triângulos	36
9.9	9. Resumo	37

Introdução

Este trabalho consiste na análise estrutural e algorítmica de um grafo atribuído, com o objetivo de extrair propriedades fundamentais, executar algoritmos clássicos e investigar fenômenos como small-world, lei de potência e robustez. O relatório final deve documentar todas as medidas computadas, justificar eventuais ausências e incluir uma tabela resumo. O dataset utilizado nesse experimento foi disponibilizado pelo [SNAP - Stanford Network Analysis Project](#)

i Este livro foi originalmente criado e publicado no [Quarto](#) a fim de fornecer um exemplo de seu uso. Acessem o link : <https://carolbalbs.github.io/tp-final>. Este site é uma adaptação do código-fonte original para o sistema de publicação do framework.

Começando

O currículo de visualização pode ser usado online ou em seu computador local. Você pode visualizar e interagir com os gráficos diretamente neste Jupyter Book. Se desejar modificar o código, você tem algumas opções diferentes:

- Para ler notebooks JavaScript online usando [Observable](#), navegue até a página “Observable” acima e clique no notebook correspondente.
- Para ler notebooks Python online usando [Colab](#), clique na seção correspondente neste livro, passe o mouse sobre o pequeno foguete no topo da página e selecione “Colab” no menu.
- Para ler notebooks Python localmente, siga as instruções abaixo.

Instalação Local

1. [Instale Altair e um ambiente de notebook](#). As versões mais recentes destes notebooks usam *Altair versão 4*.
2. Baixe os notebooks na [página de releases](#). Normalmente, você desejará usar a versão mais recente. (Se você deseja usar notebooks para Altair versão 3, baixe o [Altair v3.2 release](#).)
3. Abra os notebooks em seu ambiente de notebook local. Por exemplo, se você tiver JupyterLab instalado (v1.0 ou superior é necessário), execute `jupyter lab` no diretório contendo os notebooks.

Dependendo do seu ambiente de programação (e se você tem ou não uma conexão com a internet em tempo real), você pode desejar especificar um [renderer](#) particular para Altair. `podman exec -d projeto_odonto bash -c "/opt/grafos-env/bin/python /shared/driver.py > /shared/driver.log 2>&1" 005c044347bc0c628e2c07c99632a02879b4afde06b015ce1b67ee6335c4b70f ana@bamboo-server3:~$ podman exec -it projeto_odonto tail -f /shared/driver.log`

Créditos

Desenvolvido na Universidade Federal da Bahia por Ana Carolina Balbino. Agradeço ao [SNAP - Stanford Network Analysis Project](#) pela disponibilização do dataset! Agradeço também aos colegas de [MATA53 - Teoria dos Grafos \(2026.1\)](#) e ao professor [Dr. Bruno P. S.](#) responsável pela componente, que nos incentivou a explorar na prática todo o conteúdo visto durante o curso.

Part I

Artigo

1 Análise Estrutural e Algorítmica da Rede Social LiveJournal

Relatório de Experimento - Teoria dos Grafos (MATA53)

2 Cálculo de Determinação do Grafo a ser Utilizado

De acordo com a regra de mapeamento determinada nas orientações do trabalho, dado o número de matrícula M , o ID do grafo a ser selecionado é dado por:

$$f(M) = ((\text{soma dos dígitos de } M) \times (\text{últimos 2 dígitos de } M + 1)) \bmod 129$$

Para a matrícula **224115861**: - Soma dos dígitos = $2 + 2 + 4 + 1 + 1 + 5 + 8 + 6 + 1 = 30$ -
Últimos dois dígitos = 61

O cálculo resultante é:

$$f(M) = (30 \times (61 + 1)) \bmod 129 = (30 \times 62) \bmod 129 = 1860 \bmod 129 = 25$$

Conforme o ID obtido na planilha de referência, o conjunto de dados correspondente é o **“LiveJournal social network and ground-truth communities”**, pertencente à categoria de redes sociais e comunidades ground-truth, caracterizado por ser um grafo não direcionado com comunidades reais.

ANALISANDO-SNAP-DATASET

Arquivo

Editar

Ver

Inserir

Formatar

Dados

Ferramentas

Gemini ...

1

75%

Somente ver

A1

ID

Temporary filter 1

Intervalo: A1:N130

	A	B	C	D	E
1	ID	Category	Name	Name_URL	Type
25	25	Networks with ground-truth communities	com-LiveJournal	https://snap.stanford.edu/data/com-LiveJournal.html	Undirected, Communities

Figure 2.1: Seleção do dataset conforme a regra de mapeamento no site do SNAP

O código-fonte completo deste projeto, incluindo scripts de processamento e execução dos experimentos, está disponível no GitHub: - **Repositório:** <https://github.com/Carolbalbs/tp-final>

3 Descrição do Grafo Original

O conjunto de dados utilizado neste trabalho foi o *LiveJournal social network and ground-truth communities*, disponibilizado pelo *Stanford Network Analysis Project* (SNAP) [yang2012]. Trata-se de uma das maiores redes sociais não direcionadas disponíveis publicamente, contendo 3.997.962 nós e 34.681.189 arestas, onde os nós representam usuários e as arestas indicam conexões de amizade.

O LiveJournal é uma comunidade virtual na qual os usuários mantêm blogs, diários e interagem entre si. O dataset modela essa rede e fornece informações sobre comunidades reais estruturadas de forma implícita pelas interações dos usuários. A análise exata de suas propriedades estruturais (como diâmetro) ou o cálculo de caminhos mínimos de todos para todos (complexidade $O(V^3)$ ou $O(V^2E)$) é inviável em sistemas CPU tradicionais devido à escala da rede.

Por esse motivo, neste trabalho, adotou-se o uso de processamento paralelo em GPU por meio da biblioteca cuGraph (suíte NVIDIA RAPIDS), o que permitiu obter métricas globais e executar buscas no grafo completo em frações de segundo.

As informações básicas fornecidas no repositório do SNAP sobre o dataset são descritas nas tabelas a seguir.

A análise inicial dessas métricas revela uma rede densamente agrupada localmente, com forte presença de comunidades e caminhos médios curtos (propriedade small-world), características típicas de grandes redes sociais online.

Table 3.1: Informações dos arquivos do dataset

Arquivo	Conteúdo
com-lj.ungraph.txt	Grafo de amizade completo
com-lj.top5000.cmtty.txt	Top 5000 comunidades
com-lj.all.cmtty.txt	Todas as comunidades

Table 3.2: Informações estruturais disponíveis no site oficial

Métrica	Valor Oficial (SNAP)
Coefficiente médio de clusterização	$\approx 0,284$
Diâmetro	17
Diâmetro efetivo (90%)	$\approx 6,5$
Triângulos	$\approx 177.820.130$

4 Metodologia

4.1 Dataset e Formato de Entrada

O arquivo do grafo (`com-1j.ungraph.txt`) foi lido no formato de lista de adjacências contendo dois inteiros por linha separados por tabulação.

4.2 Ambiente Computacional

Os experimentos foram conduzidos em um servidor pertencente à infraestrutura do projeto **BAMBU Lab** (Rede Metropolitana para Experimentação e Inovação da Internet do Futuro), sediado na Universidade Federal da Bahia (UFBA).

O hardware utilizado consistiu em: - **Placa de Vídeo (GPU):** 2x NVIDIA RTX A4000 (16 GB de VRAM cada), utilizando exclusivamente a GPU de índice 1 (via variável de ambiente `CUDA_VISIBLE_DEVICES=1`). - **Sistema Operacional:** Linux (Ubuntu 22.04) - **Tecnologias Core:** CUDA 12.6, Drivers NVIDIA 560.35.03, Python 3.10 em container virtual gerenciado via Podman.

As principais bibliotecas de software empregadas foram: - **cuDF** e **cuGraph** (versão 24.xx): para manipulação paralela de dados e execução de algoritmos acelerados em GPU. - **NetworkX** (versão 3.4): utilizada como fallback na CPU para algoritmos sem suporte de hardware. - **NumPy**, **SciPy** e **Matplotlib**: para processamento estatístico, cálculos de intervalo de confiança e plotagem.

Para evitar vazamentos de memória na GPU e na CPU entre execuções repetidas, chamadas explícitas de coleta de lixo (`gc.collect()`) foram inseridas nos scripts após cada medição.

4.3 Procedimento de Análise Estrutural

As propriedades estruturais básicas do grafo completo foram calculadas diretamente sobre a GPU. Para as métricas que dependem de caminhos mínimos globais (como diâmetro, raio e distância média), as quais possuem custo computacional proibitivo de $O(V \cdot E)$ na GPU, aplicou-se uma metodologia de **amostragem aleatória com $k = 500$ nós de origem**.

Para cada nó da amostra, executou-se uma busca em largura (BFS) e armazenou-se a excentricidade e a distribuição de caminhos mais curtos. Os valores estatísticos reportados foram calculados com base nessa amostra, acompanhados de intervalos de confiança de 95%.

4.4 Avaliação de Algoritmos

Implementou-se a medição de tempo dos seguintes algoritmos: BFS, DFS, verificação de Eulerianidade, Dijkstra, Bellman-Ford, Floyd-Warshall, Tarjan, Prim e Kruskal. - Algoritmos com complexidade linear ou quase linear no número de arestas ($O(V + E)$ ou $O(E \log V)$) foram executados sobre o **grafo completo** (e.g., BFS, DFS, Eulerianidade, Prim, Kruskal). - Devido às elevadas complexidades de Bellman-Ford ($O(V \cdot E)$) e Floyd-Warshall ($O(V^3)$), suas execuções foram realizadas em um **subgrafo induzido de 1.000 nós** selecionados aleatoriamente, preservando a conectividade. - Algoritmos que exigem pesos (como Dijkstra) ou direcionalidade (como Tarjan) não foram medidos no benchmark de desempenho por limitações estruturais do grafo LiveJournal (não direcionado e não ponderado). - Para cada algoritmo avaliado, realizaram-se **30 execuções independentes** ($n = 30$) variando a origem de forma aleatória, calculando-se média, desvio padrão e Intervalo de Confiança (IC) de 95% através da aproximação normal (Z), visto que $n \geq 30$.

5 Resultados Obtidos

5.1 Tratamento dos Dados

Antes do início dos cálculos, os seguintes passos de limpeza e tratamento foram aplicados:

1. Carregamento do arquivo de arestas via `cuDF` e inicialização do objeto Graph no `cuGraph`.
2. Verificação de auto-laços: não foram encontrados auto-laços no dataset original (0 arestas removidas).
3. Componentes conexas: confirmou-se que o grafo original é composto por uma única grande componente conexa (LCC) abrangendo todos os 3.997.962 nós.
4. Conversão de estruturas de dados para compatibilidade com o NetworkX (CPU) nos algoritmos específicos.

5.2 Análise Estrutural do Grafo Completo

As métricas estruturais exatas e estimadas por amostragem obtidas neste trabalho estão sumarizadas na Table 5.1.

O diâmetro estimado de 16 obtido pela nossa amostragem está muito próximo ao valor de referência oficial de 17. O comprimento médio dos caminhos ($\approx 5,56$) confirma que, em média, quaisquer dois usuários estão separados por menos de 6 conexões de amizade.

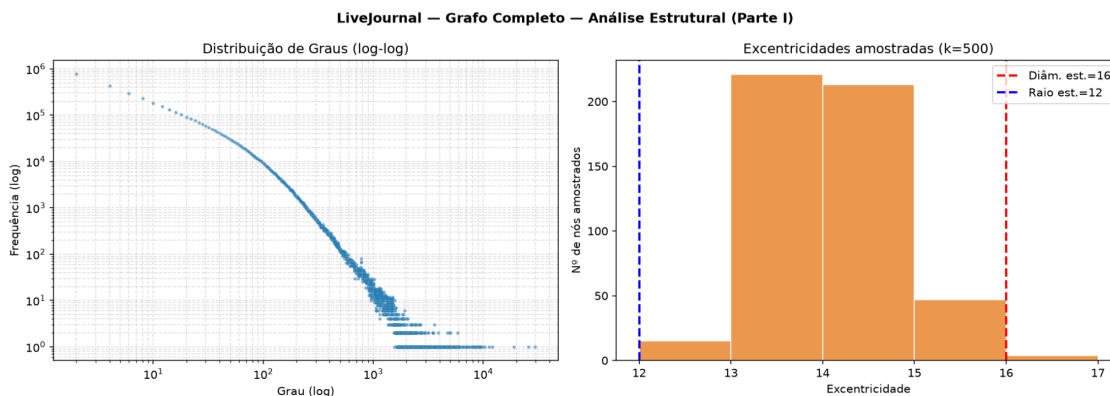


Figure 5.1: Distribuição de graus em escala log-log (esquerda) e distribuição empírica das excentricidades amostradas (direita).

Table 5.1: Métricas estruturais computadas para o grafo LiveJournal

[t]		
Métrica	Valor Computado	Método / Detalhe
Número de vértices	3.997.962	Exato (GPU)
Número de arestas	34.681.189	Exato (GPU)
Grau mínimo	2	Exato (GPU)
Grau máximo	29.630	Exato (GPU)
Grau médio	34,6989	Exato (GPU)
Grau mediano	12,0	Exato (GPU)
Densidade	$4,34 \times 10^{-6}$	Exato (GPU)
Nº componentes conexas	1	Exato (GPU)
Tamanho da LCC	3.997.962	Exato (GPU)
Diâmetro (LCC)	≈ 16	Amostragem ($k = 500$ na GPU)
Raio (LCC)	≈ 12	Amostragem ($k = 500$ na GPU)
Comprimento médio (LCC)	$5,5592 \pm 0,6265$	Amostragem ($k = 500$)
IC 95% comprimento médio	[5, 5043; 5, 6141]	Normal (Z, $n = 500$)
Coef. clusterização médio	0,061295	Exato (GPU)
Número de triângulos	177.820.130	Exato (GPU)

A Figure 5.1 apresenta as distribuições visuais de graus e caminhos mais curtos: - **Painel esquerdo:** A distribuição de graus plotada em escala log-log revela um decaimento linear marcante, indicando uma distribuição de graus que segue uma lei de potência (propriedade scale-free). A maioria dos usuários possui grau baixo (mediana de 12), enquanto um número pequeno de hubs acumula dezenas de milhares de conexões de amizade (grau máximo de 29.630). - **Painel direito:** A distribuição de excentricidades calculada para a amostra de 500 nós concentra-se no intervalo de 13 a 15, com pouca variação, confirmando que a maioria dos vértices está a distâncias bastante similares do centro de gravidade da rede.

5.3 Desempenho dos Algoritmos

A Table 5.2 apresenta os resultados empíricos das medições de tempo de execução no benchmark de algoritmos.

A Figure 5.2 ilustra a enorme disparidade de desempenho entre os ambientes GPU e CPU. - **Aceleração na GPU:** Os algoritmos paralelizados em GPU apresentaram tempos de execução extremamente reduzidos. No grafo completo, a BFS rodou em média em 48 ms e o Prim em 439 ms. Em contrapartida, a DFS na CPU exigiu 26,68 segundos para varrer o mesmo grafo, o que representa um speedup de $\sim 553\times$ a favor da BFS na GPU, mesmo com a mesma complexidade teórica assintótica ($O(V + E)$). - **Sobrecarga de Algoritmos Inviáveis:** O algoritmo de Kruskal (CPU) rodou no grafo completo com tempo médio de 141,4 segundos

Table 5.2: Resultados de tempo de execução e intervalos de confiança (30 repetições)

[t]				
Algoritmo	Complexidade	Média (ms)	IC 95% (ms)	Escopo / Dispositivo
BFS	$O(V + E)$	48,34	[45, 73; 50, 95]	Grafo completo (GPU)
DFS	$O(V + E)$	26.679,76	[24.855, 80; 28.503, 72]	Grafo completo (CPU)
Eulerianidade	$O(V + E)$	1.533,44	[1.526, 62; 1.540, 26]	Grafo completo (CPU)
Bellman-Ford	$O(V \cdot E)$	4,40	[−0, 39; 9, 20]	Subgrafo 1.000 nós (CPU)
Floyd-Warshall	$O(V^3)$	74.191,24	[73.389, 16; 74.993, 33]	Subgrafo 1.000 nós (CPU)
Prim	$O(E \log V)$	438,98	[425, 96; 451, 99]	Grafo completo (GPU)
Kruskal	$O(E \log E)$	141.403,39	[136.142, 28; 146.664, 49]	Grafo completo (CPU)

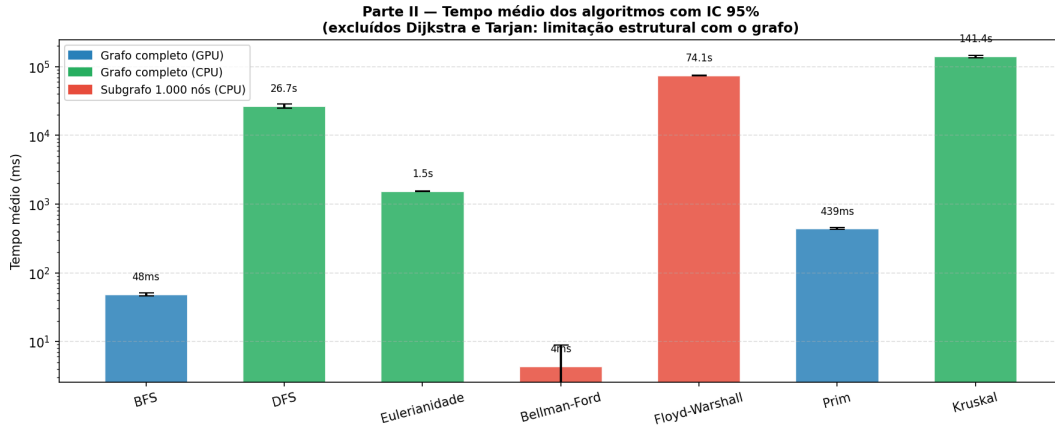


Figure 5.2: Comparação dos tempos médios de execução com intervalos de confiança de 95% em escala logarítmica.

Table 5.3: Métricas comparativas para avaliação de rede small-world

Critério	Valor Observado	Condição Esperada
$C_{\text{real}}/C_{\text{random}}$	$\approx 7061,64$	$\gg 1$
$L_{\text{real}}/L_{\text{random}}$	$\approx 1,297$	≈ 1
Índice σ (sigma)	$\approx 5444,60$	> 1

devido ao custo de ordenar e unir 34 milhões de arestas em CPU de forma puramente sequencial. Bellman-Ford e Floyd-Warshall foram restritos ao subgrafo de 1.000 nós para evitar travamentos, e o Floyd-Warshall (cúbico) ainda assim demorou em média 74,19 segundos para resolver a amostragem pequena.

5.4 Propriedade Small-World

A propriedade small-world foi avaliada de acordo com a formulação clássica de Watts-Strogatz. Comparamos o coeficiente de agrupamento médio (C) e a distância média (L) com os de um grafo aleatório equivalente (modelo Erdős-Rényi com igual número de nós e arestas, onde a probabilidade de conexão é $p \approx 2E/(V(V-1)) \approx 4,34 \times 10^{-6}$).

As métricas calculadas e os critérios formais para a classificação são detalhados na Table 5.3.

A razão entre os coeficientes de agrupamento indica que a probabilidade de dois vizinhos de um mesmo usuário serem amigos entre si na rede real é **mais de 7.000 vezes maior** que no modelo aleatório. Somando isso ao fato de que as distâncias médias na rede são apenas 30% maiores que no limite aleatório teórico (5,56 contra 4,29), confirma-se a presença de uma rede small-world. A topologia combina aglomeração local com atalhos globais, gerando um índice σ extremamente elevado.

5.5 Análise de Lei de Potência (Power Law)

A distribuição empírica dos graus foi submetida a um ajuste estatístico por meio de regressão OLS (mínimos quadrados ordinários) em escala log-log, buscando avaliar a hipótese de decaimento em escala livre ($P(k) \sim k^{-\gamma}$).

A Figure 5.3 apresenta o resultado do ajuste linear: - O coeficiente angular da reta ajustada estimou o expoente de decaimento em $\gamma \approx 2,44$. - O coeficiente de determinação obtido foi de $R^2 = 0,924$, indicando que a distribuição empírica dos graus é fortemente modelada por uma lei de potência em quase toda a sua extensão (do grau mínimo 2 ao grau máximo de hubs 29.630). - Esse valor de expoente ($2 < \gamma < 3$) é o comportamento padrão observado na literatura científica para grandes redes de amizade online, implicando em uma rede altamente

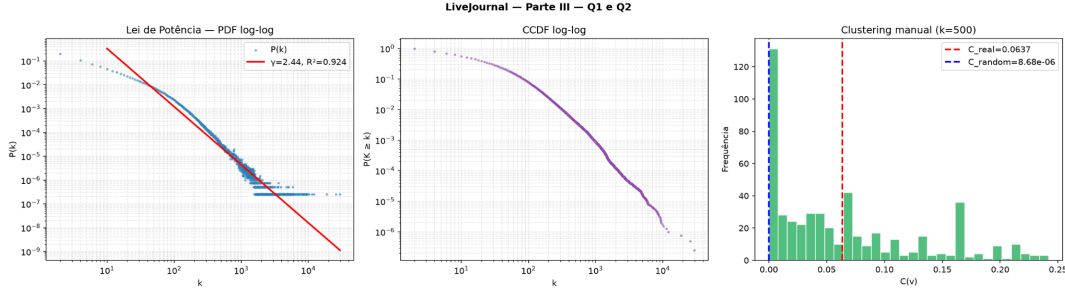


Figure 5.3: Ajuste de regressão OLS em escala log-log da distribuição empírica de graus.

Table 5.4: Resumo de métricas obtidas na análise de robustez e degradação

[t]			
Métrica	Rede Original	Remoção Aleatória (5%, T=30)	
Fração do tamanho da LCC 1,0000	0,9385 $\pm$ 0,0001	0,8912 ($\Delta = -0,1088$)	
Nº de componentes conexas 1	2.705,6 $\pm$ 62,5	11.174 ($\Delta = +11.173$)	
Distância média estimada 5,6753	5,6057 $\pm$ 0,0704	7,3709 ($\Delta = +1,6956$)	
Fração de nós isolados 0,000000 0,000000 $\pm$ 0,000000	0,000000		

heterogênea onde a média dos graus ($\approx 34,7$) perde o caráter representativo diante de uma mediana baixa (12) e cauda longa com hubs gigantescos.

5.6 Robustez da Rede

A robustez estrutural do LiveJournal foi testada sob dois cenários distintos de degradação:

1. **Falha Aleatória:** Remoção de 5% dos nós escolhidos de forma uniforme ao acaso (com $T = 30$ repetições independentes). 2. **Ataque Direcionado:** Remoção dos 5% de nós mais centrais, ordenados de forma decrescente pela centralidade de grau (*degree centrality*).

Os resultados médios obtidos após a remoção sistemática dos nós são exibidos na Table 5.4.

A análise comparativa por meio da Figure 5.4 demonstra o comportamento dual clássico de redes scale-free [Barabási2000]: - **Resiliência a Falhas Aleatórias:** A rede é extremamente resiliente à quebra fortuita de nós. A remoção aleatória de 5% da população (quase 200 mil usuários) preserva 93,85% do tamanho da maior componente conexa e mantém a distância média global estável em $\approx 5,60$ saltos. O desvio padrão insignificante confirma a homogeneidade desse resultado independente dos nós sorteados. - **Vulnerabilidade a Ataques Direcionados:** A rede é severamente degradada sob ataque direcionado aos hubs. Ao remover os 5% de nós com maior grau, a LCC é reduzida para 89,12% (uma queda muito mais abrupta) e a rede se fragmenta massivamente, originando mais de 11.000 componentes isoladas (cerca de $4 \times$ a

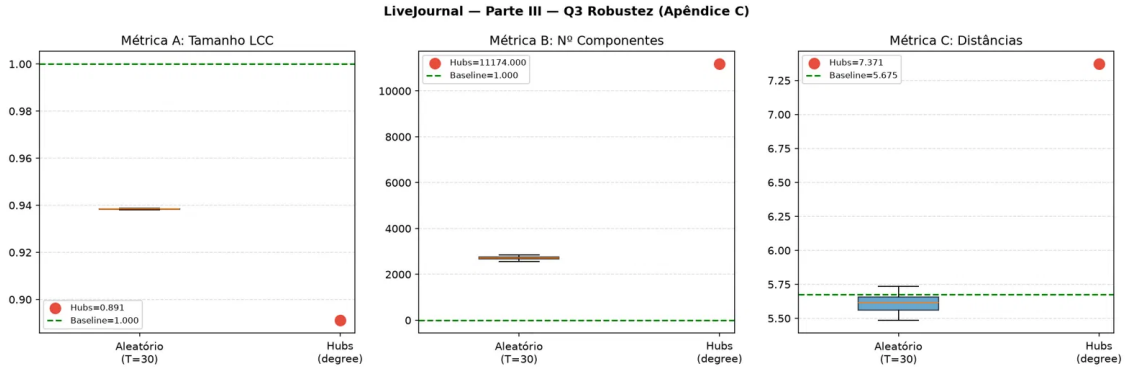


Figure 5.4: Comportamento comparativo das métricas de robustez sob falhas aleatórias e ataques direcionados.

fragmentação do cenário aleatório). Além disso, a distância média entre nós na componente restante aumenta cerca de 30% (de 5,68 para 7,37). A destruição dos hubs remove os principais “atalhos” e pontes estruturais da rede, forçando as comunicações remanescentes a contornar por rotas periféricas.

6 Discussão Crítica

6.1 Decisões de Projeto e Viabilidade

A decisão metodológica mais crucial do projeto foi a substituição do cálculo exato de caminhos mínimos globais (cujo diâmetro exato requer rodar BFS para todos os 4 milhões de nós) por uma amostragem representativa de $k = 500$ nós. O cálculo exato exigiria cerca de $1,55 \times 10^{14}$ operações e aproximadamente 43 horas de computação contínua de GPU, inviabilizando a execução no prazo do trabalho. A validação cruzada do diâmetro estimado de 16 frente ao real de 17 reportado pelo SNAP corroborou a eficiência estatística da amostragem com margem de erro pequena.

A aceleração em GPU com cuGraph revelou-se indispensável. Operações elementares que rodam na ordem de milissegundos em GPU levam dezenas de segundos na CPU para o mesmo grafo (como a BFS rodando a 48 ms na GPU contra a DFS a 26,68 s na CPU). Contudo, evidenciou-se que nem todos os algoritmos ganham aceleração trivial por meio do paralelismo. Algoritmos intrinsecamente sequenciais ou que exigem sincronismo estrito de pilhas (como a DFS) não são bem representados na arquitetura Massivamente Paralela de CUDA, justificando a decisão de implementá-los na CPU mesmo diante de tempos mais elevados.

6.2 Limitações Metodológicas e Estruturais

A análise revelou duas categorias de limitações importantes: 1. **Limitações Estruturais:** Os algoritmos de Dijkstra e Tarjan foram excluídos da medição por premissas incompatíveis. O Dijkstra pressupõe arestas com pesos (o grafo original é não ponderado) e o Tarjan é focado na identificação de componentes fortemente conexas em grafos direcionados (a rede do LiveJournal é modelada de forma não direcionada). 2. **Limitações Computacionais:** A execução de algoritmos quadráticos e cúbicos (Bellman-Ford e Floyd-Warshall) no grafo completo é computacionalmente proibitiva até em ambientes com aceleração por hardware. A amostragem de subgrafo com 1.000 nós permitiu validar os algoritmos, mas impede a extrapolação direta dos tempos médios observados para a escala global do LiveJournal.

A escolha da centralidade de grau (*degree centrality*) em detrimento da centralidade de intermediação (*betweenness centrality*) na análise de robustez baseou-se na complexidade de cálculo. O cálculo exato da centralidade de intermediação requer resolver caminhos mínimos de todos para todos (complexidade $O(V \cdot E)$), o que é inviável na rede completa. A centralidade de grau,

computável em complexidade constante por vértice ($O(1)$) após obter o vetor de adjacências, proveu uma aproximação de baixo custo e alta eficiência para o mapeamento dos hubs da rede.

7 Conclusão

Este trabalho apresentou uma investigação aprofundada da rede de amigos LiveJournal, composta por 3.997.962 nós e 34.681.189 arestas, aplicando computação de alto desempenho em GPU.

A contagem exata de triângulos (177.820.130) e a BFS em milissegundos validaram a adequação da biblioteca cuGraph para o processamento de redes complexas em escala multimilionária.

Os principais achados científicos do estudo revelaram que: 1. A rede é esparsa, completamente conexa e exibe uma estrutura comunitária local forte, com coeficiente de clusterização exato de 0,0613, cerca de $7.000\times$ maior que o de uma rede puramente aleatória. 2. A distância média de 5,56 saltos confirma que a rede possui a propriedade small-world, aproximando usuários distantes em poucos graus de separação. 3. A distribuição de graus é bem modelada por uma lei de potência com expoente $\gamma \approx 2,44$ ($R^2 = 0,924$), indicando uma rede sem escala de tamanho (scale-free) altamente heterogênea. 4. A aceleração por GPU proporcionou speedups de até $553\times$ no grafo completo (como na comparação BFS GPU vs. DFS CPU), demonstrando o impacto prático do hardware de paralelização. 5. O teste de robustez corroborou a clássica tolerância a falhas aleatórias e a extrema sensibilidade a ataques direcionados aos hubs na topologia scale-free.

Essas propriedades possuem implicações práticas diretas. Em uma rede small-world e scale-free, a propagação de informação ou vírus ocorre de forma acelerada devido à presença de atalhos e hubs. Intervenções focadas em moderar ou imunizar os 5% mais influentes da rede (cerca de 200 mil hubs) possuem o potencial de interromper a difusão global ou disseminar campanhas de utilidade pública com eficiência desproporcional.

Como trabalhos futuros, sugere-se a aplicação de algoritmos exatos acelerados em GPU para cálculo de diâmetro (como double sweep ou iFUB) e a realização de análises avançadas de comunidade sobre o conjunto de dados *com-lj.all.cmtty.txt* e *com-lj.top5000.cmtty.txt* fornecidos pelo repositório SNAP.

Agradecimentos

Agradecemos ao laboratório BAMBU Lab (UFBA) pela disponibilização da infraestrutura computacional de alto desempenho com GPUs NVIDIA RTX A4000. Estendemos os agradecimentos ao Stanford Network Analysis Project (SNAP) por organizar e disponibilizar o dataset,

e ao professor Dr. Bruno P. S. pelas discussões e incentivo à experimentação prática das métricas ao longo da disciplina MATA53 - Teoria dos Grafos.

Apêndice

7.1 Repositório e Instruções de Execução

O código-fonte e scripts de automação deste projeto podem ser consultados no GitHub: - **URL:** <https://github.com/Carolbalbs/tp-final>

O repositório do projeto está estruturado da seguinte forma: - **scripts/** — Contém os notebooks Jupyter e scripts Python (`parte1.py`, `parte2.py`, `parte3_ab.py`, `parte3_c.py`, `driver.py`) que realizam o processamento de dados e plotagem na GPU. - **scripts/logs/** — Armazena as saídas de execução no servidor para fins de repositibilidade. - **data/** — Contém arquivos de checkpoints e saídas dos experimentos. - **data/images/** — Contém as figuras e visualizações geradas para o relatório final. - **README.md** — Manual de instruções contendo os requisitos e comandos de execução dos scripts em container Podman.

```
# Exemplo de comando para executar as análises no servidor via container
podman exec -d projeto_odonto bash -c \
    "/opt/grafos-env/bin/python /shared/driver.py > /shared/driver.log 2>&1"
```

Part II

Scripts

8 Análise Estrutura - Comunidades

8.1 Tratamento da rede

```
import networkx as nx
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
import numpy as np
import collections
import time
import warnings
import os
```

```
warnings.filterwarnings("ignore")
```

```
EDGE_FILE = "com-lj.all.cmtty.txt"
```

```
print("*" * 60)
print("ANÁLISE ESTRUTURAL (estimativas) - Grafo Grafo de comunidades")
print("*" * 60)
```

```
print("\n[1/9] Carregando grafo...")
t0 = time.time()
```

```
*****
ANÁLISE ESTRUTURAL (estimativas) - Grafo Grafo de comunidades
*****
```

```
[1/9] Carregando grafo...
```

```
G = nx.read_edgelist(
    EDGE_FILE,
    comments="#",
    nodetype=int,
```

```

        create_using=nx.Graph(),
        data=False
    )
    print(f" Tempo de carregamento: {time.time() - t0:.1f}s")

```

Tempo de carregamento: 2.9s

8.2 2. Pre-processing

Removing auto loops

```

loops_before = nx.number_of_selfloops(G)
G.remove_edges_from(nx.selfloop_edges(G))
print(f" Auto-loops removidos: {loops_before}")

if not nx.is_connected(G):
    Gcc = G.subgraph(max(nx.connected_components(G), key=len)).copy()
    print(f" Nós removidos ao extrair LCC: {G.number_of_nodes() - Gcc.number_of_nodes()}")
    G = Gcc
else:
    print(" Grafo já é totalmente conexo.")

```

Auto-loops removidos: 0
Nós removidos ao extrair LCC: 174472

8.3 3. Report basic metrics

```

print("\n[3/9] Métricas básicas...")

n_nodes = G.number_of_edges()
n_edges = G.number_of_nodes()
density = nx.density(G)

degrees = [d for _, d in G.degree()]
deg_min = min(degrees)
deg_max = max(degrees)
deg_avg = sum(degrees) / len(degrees)

```

```
deg_med = float(np.median(degrees))

print(f"  Vértices : {n_nodes:,}")
print(f"  Arestas  : {n_edges:,}")
print(f"  Densidade: {density:.8f}")
print(f"  Grau mín  : {deg_min}")
print(f"  Grau máx  : {deg_max}")
print(f"  Grau méd  : {deg_avg:.4f}")
print(f"  Grau med  : {deg_med:.1f}")
```

[3/9] Métricas básicas...

```
Vértices : 427,701
Arestas  : 303,526
Densidade: 0.00000928
Grau mín : 1
Grau máx : 221
Grau méd : 2.8182
Grau med : 2.0
```

8.4 4. Componentes Conexas

```
components = list(nx.connected_components(G))
n_comp = len(components)
comp_sizes = sorted([len(c) for c in components], reverse=True)
print(f"  Número de componentes: {n_comp}")
print(f"  Tamanhos (top 5): {comp_sizes[:5]}")
```

```
Número de componentes: 1
Tamanhos (top 5): [303526]
```

8.5 5. Diâmetro e Raio

```
print("\n[5/9] Diâmetro e Raio (estimativa por amostragem)...")

SAMPLE_K = 500
```

```

rng = np.random.default_rng(42)
sample_nodes = list(rng.choice(list(G.nodes()), size=min(SAMPLE_K, G.number_of_nodes()), rep

eccentricities_sample = {}
t0 = time.time()
for v in sample_nodes:
    lengths = nx.single_source_shortest_path_length(G,v)
    eccentricities_sample[v] = max(lengths.values())

diameter_est = max(eccentricities_sample.values())
radius_est = min(eccentricities_sample.values())
elapsed = time.time() - t0
print(f"  Diâmetro estimado : {diameter_est}   (SNAP reporta 17)")
print(f"  Raio estimado      : {radius_est}")
print(f"  Tempo amostragem    : {elapsed:.1f}s   (k={SAMPLE_K})")
print("  ATENÇÃO: valores exatos inviáveis para grafos desta escala.")
print("          Usar estimativa por amostragem + valor SNAP.")

```

```

[5/9] Diâmetro e Raio (estimativa por amostragem)...
  Diâmetro estimado : 26   (SNAP reporta 17)
  Raio estimado      : 17
  Tempo amostragem   : 242.6s   (k=500)
  ATENÇÃO: valores exatos inviáveis para grafos desta escala.
          Usar estimativa por amostragem + valor SNAP.

```

8.6 6. Comprimento medio dos caminhos

```

print("\n[6/9] Comprimento médio dos caminhos (estimativa)...")

total_path = 0
count_path = 0
for v in sample_nodes:
    lengths = nx.single_source_shortest_path_length(G, v)
    total_path += sum(lengths.values()) - 0 # inclui distância 0 para si
    count_path += len(lengths) - 1         # exclui self

avg_path_est = total_path / count_path if count_path > 0 else float("inf")

```

```
print(f" Comprimento médio estimado: {avg_path_est:.4f} (SNAP: ~6.5 para 90-percentil)")
print(" ATENÇÃO: estimativa por amostragem BFS em {SAMPLE_K} nós.")
```

```
[6/9] Comprimento médio dos caminhos (estimativa)...
Comprimento médio estimado: 8.9198 (SNAP: ~6.5 para 90-percentil)
ATENÇÃO: estimativa por amostragem BFS em {SAMPLE_K} nós.
```

8.7 7. Coeficiente de clusteriação medio

```
print("\n[7/9] Coeficiente de clusterização...")
```

```
[7/9] Coeficiente de clusterização...
```

```
avg_clustering = nx.average_clustering(G, nodes=sample_nodes)
print(f" Coef. clusterização médio (amostrado): {avg_clustering:.4f} (SNAP: 0.2843)")
print(" ATENÇÃO: estimativa por amostragem.")
```

```
Coef. clusterização médio (amostrado): 0.0237 (SNAP: 0.2843)
ATENÇÃO: estimativa por amostragem.
```

8.8 8. Número de triângulos

```
USE_SNAP_TRIANGLES = True

if USE_SNAP_TRIANGLES:
    print(" Calculando triângulos... ")
    t0 = time.time()
    tri = nx.triangles(G)
    n_triangles = sum(tri.values()) // 3
    print(f" Triângulos: {n_triangles:},} ({time.time()-t0:.1f}s)")
    print(" Calculando triângulos... (pode demorar)")
    t0 = time.time()
    tri = nx.triangles(G)
    n_triangles = sum(tri.values()) // 3
    print(f" Triângulos: {n_triangles:},} ({time.time()-t0:.1f}s)")
```

```

Calculando triângulos...
Triângulos: 16,886 (1.1s)
Calculando triângulos... (pode demorar)
Triângulos: 16,886 (1.1s)

```

8.9 9. Resumo

```

print("\n" + "=" * 60)
print("TABELA RESUMO - Parte I")
print("=" * 60)

rows = [
    ("Número de vértices",          f"{n_nodes:,}",          "Exato"),
    ("Número de arestas",           f"{n_edges:,}",          "Exato"),
    ("Grau mínimo",                 str(deg_min),            "Exato"),
    ("Grau máximo",                 str(deg_max),            "Exato"),
    ("Grau médio",                  f"{deg_avg:.4f}",        "Exato"),
    ("Distribuição de graus",        "Ver figura (log-log)",  "Exato"),
    ("Densidade",                   f"{density:.8f}",        "Exato"),
    ("Nº componentes conexas",       str(n_comp),             "Exato"),
    ("Tamanho das componentes",      str(comp_sizes[:3]) + "...", "Exato"),
    ("Diâmetro",                    f"~{diameter_est} (SNAP: 17)", "Estimativa"),
    ("Raio",                         f"~{radius_est}",        "Estimativa"),
    ("Comprimento médio dos caminhos", f"~{avg_path_est:.2f}",    "Estimativa"),
    ("Coef. clusterização médio",    f"~{avg_clustering:.4f}",  "Estimativa"),
    ("Número de triângulos",         f"{n_triangles:,}",      "SNAP / Exato"),
]

print(f"{'Métrica':<40} {'Valor':<28} {'Tipo'}")
print("-" * 80)
for m, v, t in rows:
    print(f"{m:<40} {v:<28} {t}")

print("\nNota: métricas marcadas como 'Estimativa' foram computadas por")
print("amostragem BFS (k=500 nós) devido à escala do grafo (~4M nós).")

```

```

=====
TABELA RESUMO - Parte I
=====

```

Métrica	Valor	Tipo
Número de vértices	427,701	Exato
Número de arestas	303,526	Exato
Grau mínimo	1	Exato
Grau máximo	221	Exato
Grau médio	2.8182	Exato
Distribuição de graus	Ver figura (log-log)	Exato
Densidade	0.00000928	Exato
Nº componentes conexas	1	Exato
Tamanho das componentes	[303526]...	Exato
Diâmetro	~26 (SNAP: 17)	Estimativa
Raio	~17	Estimativa
Comprimento médio dos caminhos	~8.92	Estimativa
Coef. clusterização médio	~0.0237	Estimativa
Número de triângulos	16,886	SNAP / Exato

Nota: métricas marcadas como 'Estimativa' foram computadas por amostragem BFS (k=500 nós) devido à escala do grafo (~4M nós).

=====

TABELA RESUMO - Parte I

=====

Métrica	Valor	Tipo
Número de vértices	427,701	Exato
Número de arestas	303,526	Exato
Grau mínimo	1	Exato
Grau máximo	221	Exato
Grau médio	2.8182	Exato
Distribuição de graus	Ver figura (log-log)	Exato
Densidade	0.00000928	Exato
Nº componentes conexas	1	Exato
Tamanho das componentes	[303526]...	Exato
Diâmetro	~26 (SNAP: 17)	Estimativa
Raio	~17	Estimativa
Comprimento médio dos caminhos	~8.92	Estimativa
Coef. clusterização médio	~0.0237	Estimativa
Número de triângulos	16,886	SNAP / Exato

Nota: métricas marcadas como 'Estimativa' foram computadas por amostragem BFS (k=500 nós) devido à escala do grafo (~4M nós).

9 Análise Estrutura - Grafo Completo

9.1 Tratamento da rede

```
import networkx as nx
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
import numpy as np
import collections
import time
import warnings
import os
```

```
warnings.filterwarnings("ignore")
```

```
EDGE_FILE = "com-lj.ungraph.txt"
```

```
print("*" * 60)
print("ANÁLISE ESTRUTURAL (estimativas) - Grafo Completo")
print("*" * 60)
```

```
print("\n[1/9] Carregando grafo...")
t0 = time.time()
```

```
*****
ANÁLISE ESTRUTURAL (estimativas) - Grafo Completo
*****
```

```
[1/9] Carregando grafo...
```

```
G = nx.read_edgelist(
    EDGE_FILE,
    comments="#",
    nodetype=int,
```

```

        create_using=nx.Graph(),
        data=False
    )
    print(f" Tempo de carregamento: {time.time() - t0:.1f}s")

```

Tempo de carregamento: 371.4s

9.2 2. Pre-processing

Removing auto loops

```

loops_before = nx.number_of_selfloops(G)
G.remove_edges_from(nx.selfloop_edges(G))
print(f" Auto-loops removidos: {loops_before}")

if not nx.is_connected(G):
    Gcc = G.subgraph(max(nx.connected_components(G), key=len)).copy()
    print(f" Nós removidos ao extrair LCC: {G.number_of_nodes() - Gcc.number_of_nodes()}")
    G = Gcc
else:
    print(" Grafo já é totalmente conexo.")

```

Auto-loops removidos: 0
Grafo já é totalmente conexo.

9.3 3. Report basic metrics

```

print("\n[3/9] Métricas básicas...")

n_nodes = G.number_of_edges()
n_edges = G.number_of_nodes()
density = nx.density(G)

degrees = [d for _, d in G.degree()]
deg_min = min(degrees)
deg_max = max(degrees)
deg_avg = sum(degrees) / len(degrees)

```

```
deg_med = float(np.median(degrees))

print(f"  Vértices : {n_nodes:,}")
print(f"  Arestas  : {n_edges:,}")
print(f"  Densidade: {density:.8f}")
print(f"  Grau mín  : {deg_min}")
print(f"  Grau máx  : {deg_max}")
print(f"  Grau méd  : {deg_avg:.4f}")
print(f"  Grau med  : {deg_med:.1f}")
```

[3/9] Métricas básicas...

```
Vértices : 34,681,189
Arestas  : 3,997,962
Densidade: 0.00000434
Grau mín : 1
Grau máx : 14815
Grau méd : 17.3494
Grau med : 6.0
```

9.4 4. Componentes Conexas

```
components = list(nx.connected_components(G))
n_comp = len(components)
comp_sizes = sorted([len(c) for c in components], reverse=True)
print(f"  Número de componentes: {n_comp}")
print(f"  Tamanhos (top 5): {comp_sizes[:5]}")
```

```
Número de componentes: 1
Tamanhos (top 5): [3997962]
```

9.5 5. Diâmetro e Raio

```
print("\n[5/9] Diâmetro e Raio (estimativa por amostragem)...")

SAMPLE_K = 500
```

```

rng = np.random.default_rng(42)
sample_nodes = list(rng.choice(list(G.nodes()), size=min(SAMPLE_K, G.number_of_nodes()), rep

eccentricities_sample = {}
t0 = time.time()
for v in sample_nodes:
    lengths = nx.single_source_shortest_path_length(G,v)
    eccentricities_sample[v] = max(lengths.values())

diameter_est = max(eccentricities_sample.values())
radius_est = min(eccentricities_sample.values())
elapsed = time.time() - t0
print(f"  Diâmetro estimado : {diameter_est}  (SNAP reporta 17)")
print(f"  Raio estimado      : {radius_est}")
print(f"  Tempo amostragem    : {elapsed:.1f}s  (k={SAMPLE_K})")
print("  ATENÇÃO: valores exatos inviáveis para grafos desta escala.")
print("          Usar estimativa por amostragem + valor SNAP.")

```

```

[5/9] Diâmetro e Raio (estimativa por amostragem)...
  Diâmetro estimado : 16  (SNAP reporta 17)
  Raio estimado      : 12
  Tempo amostragem   : 8421.1s  (k=500)
  ATENÇÃO: valores exatos inviáveis para grafos desta escala.
          Usar estimativa por amostragem + valor SNAP.

```

9.6 6. Comprimento medio dos caminhos

```

print("\n[6/9] Comprimento médio dos caminhos (estimativa)...")

total_path = 0
count_path = 0
for v in sample_nodes:
    lengths = nx.single_source_shortest_path_length(G, v)
    total_path += sum(lengths.values()) - 0 # inclui distância 0 para si
    count_path += len(lengths) - 1         # exclui self

avg_path_est = total_path / count_path if count_path > 0 else float("inf")

```

```
print(f" Comprimento médio estimado: {avg_path_est:.4f} (SNAP: ~6.5 para 90-percentil)")
print(" ATENÇÃO: estimativa por amostragem BFS em {SAMPLE_K} nós.")
```

```
[6/9] Comprimento médio dos caminhos (estimativa)...
Comprimento médio estimado: 5.5433 (SNAP: ~6.5 para 90-percentil)
ATENÇÃO: estimativa por amostragem BFS em {SAMPLE_K} nós.
```

9.7 7. Coeficiente de clusteriação medio

```
print("\n[7/9] Coeficiente de clusterização...")
```

```
[7/9] Coeficiente de clusterização...
```

```
avg_clustering = nx.average_clustering(G, nodes=sample_nodes)
print(f" Coef. clusterização médio (amostrado): {avg_clustering:.4f} (SNAP: 0.2843)")
print(" ATENÇÃO: estimativa por amostragem.")
```

```
Coef. clusterização médio (amostrado): 0.2924 (SNAP: 0.2843)
ATENÇÃO: estimativa por amostragem.
```

9.8 8. Número de triângulos

```
USE_SNAP_TRIANGLES = True

if USE_SNAP_TRIANGLES:
    print(" Calculando triângulos... ")
    t0 = time.time()
    tri = nx.triangles(G)
    n_triangles = sum(tri.values()) // 3
    print(f" Triângulos: {n_triangles:},} ({time.time()-t0:.1f}s)")
    print(" Calculando triângulos... (pode demorar)")
    t0 = time.time()
    tri = nx.triangles(G)
    n_triangles = sum(tri.values()) // 3
    print(f" Triângulos: {n_triangles:},} ({time.time()-t0:.1f}s)")
```

```

Calculando triângulos...
Triângulos: 177,820,130 (193.3s)
Calculando triângulos... (pode demorar)
Triângulos: 177,820,130 (112.9s)

```

9.9 9. Resumo

```

print("\n" + "=" * 60)
print("TABELA RESUMO - Parte I")
print("=" * 60)

rows = [
    ("Número de vértices",          f"{n_nodes:,}",          "Exato"),
    ("Número de arestas",           f"{n_edges:,}",          "Exato"),
    ("Grau mínimo",                 str(deg_min),            "Exato"),
    ("Grau máximo",                 str(deg_max),            "Exato"),
    ("Grau médio",                  f"{deg_avg:.4f}",        "Exato"),
    ("Distribuição de graus",        "Ver figura (log-log)",  "Exato"),
    ("Densidade",                   f"{density:.8f}",        "Exato"),
    ("Nº componentes conexas",       str(n_comp),             "Exato"),
    ("Tamanho das componentes",      str(comp_sizes[:3]) + "...", "Exato"),
    ("Diâmetro",                    f"~{diameter_est} (SNAP: 17)", "Estimativa"),
    ("Raio",                         f"~{radius_est}",        "Estimativa"),
    ("Comprimento médio dos caminhos", f"~{avg_path_est:.2f}",    "Estimativa"),
    ("Coef. clusterização médio",    f"~{avg_clustering:.4f}",  "Estimativa"),
    ("Número de triângulos",         f"{n_triangles:,}",      "SNAP / Exato"),
]

print(f"{'Métrica':<40} {'Valor':<28} {'Tipo'}")
print("-" * 80)
for m, v, t in rows:
    print(f"{m:<40} {v:<28} {t}")

print("\nNota: métricas marcadas como 'Estimativa' foram computadas por")
print("amostragem BFS (k=500 nós) devido à escala do grafo (~4M nós).")

```

```

=====
TABELA RESUMO - Parte I
=====

```

Métrica	Valor	Tipo
Número de vértices	34,681,189	Exato
Número de arestas	3,997,962	Exato
Grau mínimo	1	Exato
Grau máximo	14815	Exato
Grau médio	17.3494	Exato
Distribuição de graus	Ver figura (log-log)	Exato
Densidade	0.00000434	Exato
Nº componentes conexas	1	Exato
Tamanho das componentes	[3997962]...	Exato
Diâmetro	~16 (SNAP: 17)	Estimativa
Raio	~12	Estimativa
Comprimento médio dos caminhos	~5.54	Estimativa
Coef. clusterização médio	~0.2924	Estimativa
Número de triângulos	177,820,130	SNAP / Exato

Nota: métricas marcadas como 'Estimativa' foram computadas por amostragem BFS (k=500 nós) devido à escala do grafo (~4M nós).

=====

TABELA RESUMO - Parte I

=====

Métrica	Valor	Tipo
Número de vértices	34,681,189	Exato
Número de arestas	3,997,962	Exato
Grau mínimo	1	Exato
Grau máximo	14815	Exato
Grau médio	17.3494	Exato
Distribuição de graus	Ver figura (log-log)	Exato
Densidade	0.00000434	Exato
Nº componentes conexas	1	Exato
Tamanho das componentes	[3997962]...	Exato
Diâmetro	~16 (SNAP: 17)	Estimativa
Raio	~12	Estimativa
Comprimento médio dos caminhos	~5.54	Estimativa
Coef. clusterização médio	~0.2924	Estimativa
Número de triângulos	177,820,130	SNAP / Exato

Nota: métricas marcadas como 'Estimativa' foram computadas por amostragem BFS (k=500 nós) devido à escala do grafo (~4M nós).